

06 · Volatility targeting

Quant-Backtester · material de estudio

Este capítulo es una extensión opcional. Hasta ahora, una posición de 1 significa "el capital asignado a ese activo, entero", sea el activo tranquilo o salvaje. Con *volatility targeting* escalas cada posición, día a día, para que el riesgo de cada activo apunte a una volatilidad anual fija: un 10 % por defecto (`config.VOL_TARGET`). Verás por qué tiene sentido, cómo estimar la volatilidad sin mirar el futuro y qué efectos secundarios tiene sobre el turnover y los costes.

Cubre un único TODO, el **6.1** `apply_vol_target`, en `src/backtest.py`. Una vez hecho, `run_backtest(..., vol_target=0.10)` lo aplica automáticamente. Necesitas el motor (TODO 2.1 a 2.6) y `simple_returns` (TODO 1.1).

1. Intuición

Riesgo comparable entre activos. En la cartera equiponderada del proyecto, cada activo recibe 1/N del capital, pero no 1/N del riesgo. Una acción individual suele tener una volatilidad anual del orden del 25-35 %; un ETF de oro o de bonos largos, del orden del 15 %. Con la misma posición, la acción oscila el doble que el ETF y los resultados de la cartera los dominan los activos más volátiles. Si divides cada posición por la volatilidad de su activo, todos aportan un riesgo parecido: iguales en riesgo, no en dinero.

Clustering de volatilidad. Los rendimientos diarios son casi impredecibles, pero su tamaño no: los días agitados vienen en racimos (otoño de 2008, marzo de 2020) y los tranquilos también. La volatilidad de las últimas semanas es una predicción razonable de la de los próximos días. Por eso funciona escalar con una estimación reciente: cuando la volatilidad sube, la posición baja, y el riesgo efectivo se mantiene cerca del objetivo.

Qué no hace. No predice la dirección del precio. Y si la escala fuera constante, no cambiaría el Sharpe: multiplicar los rendimientos por una constante multiplica por lo mismo la media y la desviación típica. Cualquier cambio de Sharpe viene de que la escala *varía* en el tiempo: si los periodos de volatilidad alta no se compensan con rendimientos proporcionalmente mayores, reducir la exposición en ellos mejora el cociente (Moreira y Muir, 2017, lo documentan en renta variable; el efecto es discutido y depende del activo). Los drawdowns suelen reducirse, porque las crisis llegan con volatilidad alta, pero hay un punto débil: la estimación mira hacia atrás. Tras un periodo muy tranquilo la escala está alta justo cuando llega un golpe repentino, y la estimación tarda días en reaccionar.



Volatilidad móvil de 60 días de un activo antes y después de escalar. Sin escalar, sigue los regímenes del mercado; escalada, vuelve al objetivo del 10 % tras cada cambio. En los cambios bruscos la estimación llega tarde: cuando la volatilidad salta, la posición escalada se pasa del objetivo durante unas semanas, y cuando cae, se queda por debajo.

2. Formalización

Notación, la del docstring: r_t es el rendimiento simple del activo el día t (el de `simple_returns`, con la primera fila NaN); w_t es la posición **ya desplazada** del día t , decidida al cierre de $t-1$; L es `lookback`; σ^* es `target_vol`; λ es `max_leverage`; y 252 es `periods`. La desviación típica muestral (ddof = 1) de L valores es

$$s(x_1, \dots, x_L) = \sqrt{\frac{1}{L-1} \sum_{i=1}^L (x_i - \bar{x})^2}$$

La volatilidad anualizada que se estima para el día t usa los L rendimientos **anteriores** a t :

$$\hat{\sigma}_t = s(r_{t-L}, \dots, r_{t-1}) \sqrt{252}$$

La escala y la posición escalada (w'_t en el docstring; aquí la llamamos v_t):

$$e_t = \min\left(\frac{\sigma^*}{\hat{\sigma}_t}, \lambda\right)$$

$$v_t = w_t \cdot e_t$$

Si $|w_t| = 1$ y la estimación acierta ($\hat{\sigma}_t \approx \sigma_t$), el P&L del activo tiene la volatilidad objetivo:

$$\sigma(v_t r_t) \approx e_t \sigma_t \approx \sigma^*$$

Solo información hasta $t-1$. La posición w_t se decide al cierre de $t-1$ y gana r_t (capítulo 2). En ese momento conoces r_{t-1} , pero no r_t . Una ventana móvil de pandas que acaba en la fila t **incluye** r_t ; la estimación que puede usar la fila t es la que esa ventana calculó en la fila $t-1$. Las posiciones, en cambio, no se tocan: ya vienen desplazadas.

Calentamiento. Mientras no haya L rendimientos válidos entre r_{t-L} y r_{t-1} , no hay $\hat{\sigma}_t$ y la escala es 0 (fuera de mercado). Como r_0 es NaN, el primer bloque completo es r_1 ,

..., r_L , y el primer día que puede usarlo es $t = L + 1$. Las filas 0 a L ($L + 1$ filas) valen 0; con $L = 60$, la primera escala positiva está en la fila 61. Ejemplo con $L = 3$ y $\sigma^* = 10\%$:

fila t	r_t	rendimientos que usa $\hat{\sigma}_t$	$\hat{\sigma}_t$	e_t
0	NaN	menos de 3 anteriores	—	0
1	+1 %	menos de 3 anteriores	—	0
2	-1 %	menos de 3 anteriores	—	0
3	0 %	r_0, r_1, r_2 (r_0 es NaN)	—	0
4	+3 %	$r_1, r_2, r_3 = +1\%, -1\%, 0\%$	15,87 %	0,630
5	...	$r_2, r_3, r_4 = -1\%, 0\%, +3\%$	33,05 %	0,303

Comprobación de la fila 4: la media de (0,01; -0,01; 0) es 0, la suma de cuadrados es 0,0002, entre $L - 1 = 2$ da 0,0001 y su raíz es $s = 0,01$. Anualizada, $0,01 \times \sqrt{252} = 0,1587$, y la escala es $0,10 / 0,1587 = 0,630$. Fila 5: la media es 0,00667, la suma de cuadrados de las desviaciones 0,000867, entre 2 da 0,000433, $s = 0,02082$, $\hat{\sigma} = 0,3305$ y $e = 0,303$. Fíjate en que el +3 % del día 4 no afecta a e_4 , solo a e_5 : es exactamente lo que comprueba `test_apply_vol_target_uses_only_past_returns`.

Con $ddof = 0$, que es lo que usa `np.std` por defecto, la fila 4 daría $s = 0,00816$ y $e = 0,771$: un 22 % más de posición. Con $L = 60$ la diferencia es mucho menor (el factor es $\sqrt{(60/59)} \approx 1,008$), pero la convención del proyecto, como en `metrics.py`, es $ddof = 1$.

Tope. Si $\hat{\sigma}_t$ es muy pequeña, el cociente se dispara, y λ lo limita. Algunos valores con $\sigma^* = 10\%$ y $\lambda = 2$: $\hat{\sigma} = 28\%$ da 0,357; $\hat{\sigma} = 16\%$ da 0,625; $\hat{\sigma} = 4\%$ daría 2,5 y se queda en 2. Si $\hat{\sigma}_t = 0$ (un activo que no se ha movido en L días), el cociente es infinito y la escala es λ .

README frente a código

En las limitaciones, el README dice "sin apalancamiento". Con esta extensión la escala puede llegar a `MAX_LEVERAGE = 2.0`, así que una posición de 1 puede convertirse en 2: apalancamiento. Manda el código; ten en cuenta que el modelo no cobra la financiación de la parte que supera 1, ni el préstamo de títulos de los cortos.

Turnover y costes. Con señal constante ($w = 1$), el turnover deja de ser 0, porque la escala cambia un poco cada día:

$$\tau_t = |w_t e_t - w_{t-1} e_{t-1}|$$

En el ejemplo, el turnover del día 4 es 0,630 (se entra desde 0) y el del día 5, $|0,303 - 0,630| = 0,327$, exagerado porque $L = 3$. Con $L = 60$ los cambios diarios son pequeños, pero constantes: si la escala se mueve de media un 1 % al día, son $252 \times 0,01 \approx 2,5$ unidades de turnover al año y, con 5 + 5 pb, unos $2,5 \times 0,001 = 0,25\%$ anual de costes. En sentido contrario, cuando la escala es menor que 1, las entradas y salidas de la señal mueven menos capital y cuestan menos. El efecto neto depende de la estrategia: buy-and-hold pasa de casi no operar a operar todos los días; una estrategia con mucho turnover de señal puede acabar pagando menos. Si los costes importan, lo habitual es alargar la ventana o rebalancear solo cuando la escala cambia más de un umbral (ningún test lo pide).

Por activo frente a por cartera. Aquí la escala se calcula activo a activo: cada P&L individual apunta al 10 %. La cartera equiponderada divide entre N y diversifica, así que su volatilidad sale por debajo. Si los N activos están siempre invertidos con la volatilidad objetivo y su correlación media es $\bar{\rho}$:

$$\sigma_p \approx \sigma^* \sqrt{\frac{1}{N} + (1 - \frac{1}{N})\bar{\rho}}$$

Con $N = 10$ y $\sigma^* = 10\%$: $\bar{\rho} = 0$ da un 3,2 %; $\bar{\rho} = 0,3$, un 6,1 %; $\bar{\rho} = 0,5$, un 7,4 %. Solo con $\bar{\rho} = 1$ llegarías al 10 %. Y si la estrategia está fuera de mercado en algunos activos, todavía menos. Para que la **cartera** tuviera un 10 % habría que estimar la volatilidad de los rendimientos de la cartera y escalar todas las posiciones con un único factor: es otra herramienta con otro objetivo, y no es la de este TODO.

3. Del papel al código

Cómo se usa. `run_backtest(prices, signals, vol_target=0.10)` convierte las señales en posiciones con el desfase de siempre, llama a `apply_vol_target` con `simple_returns(prices)`, `lookback=config.VOL_LOOKBACK (60)` y `max_leverage=config.MAX_LEVERAGE (2,0)` y hace el backtest con las posiciones escaladas. Sin `vol_target` (por defecto `None`) no cambia nada. A `run_backtest` no puedes pasarle otro `lookback` ni otro tope: lee `config.VOL_LOOKBACK` y `config.MAX_LEVERAGE` en cada llamada, así que puedes cambiarlos en `src/config.py` (o asignarlos desde el notebook), o bien llamar tú a `apply_vol_target` y luego a `backtest_positions`. El walk-forward del capítulo 05 no aplica vol targeting.

TODO 6.1 · apply_vol_target

Qué hace. Recibe las posiciones ya desplazadas y los rendimientos simples de los activos (mismo índice y mismas columnas) y devuelve las posiciones multiplicadas por una escala por activo y día, según las fórmulas del apartado 2. El resultado tiene el mismo índice y las mismas columnas que `positions`.

Firma.

```
def apply_vol_target(
    positions: pd.DataFrame,
    asset_returns: pd.DataFrame,
    target_vol: float = config.VOL_TARGET,
    lookback: int = config.VOL_LOOKBACK,
    max_leverage: float = config.MAX_LEVERAGE,
    periods: int = config.TRADING_DAYS,
) -> pd.DataFrame:
```

Convenciones que comprueban los tests. Todos los tests de `tests/test_vol_target.py` llevan la marca `extension`.

- Con dos activos sintéticos de volatilidad 30 %, objetivo 10 %, $L = 60$ y tope 5, la volatilidad anualizada del P&L escalado desde la fila 100 está a menos de un 10 % relativo del objetivo (`test_apply_vol_target_hits_target`).
- Las filas 0 a `lookback` valen 0 y la fila `lookback + 1` ya es positiva (`test_apply_vol_target_warmup_is_zero`).
- Multiplicar por 10 el rendimiento de la fila 500 no cambia ninguna escala hasta la fila 500 incluida, y sí la de la 501 (`test_apply_vol_target_uses_only_past_returns`).
- Con volatilidad muy baja (σ diaria de 0,05 %, menos del 1 % anual) o nula (rendimientos 0), la escala es exactamente `max_leverage` desde la fila 21 con $L = 20$ (`test_apply_vol_target_caps_leverage`).
- Las posiciones cortas siguen siendo ≤ 0 : la escala nunca es negativa (`test_apply_vol_target_keeps_direction`).

- `run_backtest(..., vol_target=0.10)` cambia las posiciones y su equity no tiene NaN (`test_run_backtest_with_vol_target`). Esto exige que tu salida no tenga NaN ni infinitos, porque `backtest_positions` los rechaza.

Pseudocódigo.

1. Para cada activo y cada fila `t`: desviación típica muestral (`ddof = 1`) de los `'lookback'` rendimientos que terminan en `t`, incluido. Si la ventana no tiene `'lookback'` valores válidos, NaN.
2. Pásala a la fila siguiente: la fila `t` debe contener la estimación calculada en `t - 1`.
3. Anualiza: multiplica por la raíz cuadrada de `'periods'`.
4. `escala = target_vol / volatilidad anualizada` (si la volatilidad es 0, sale infinito)
5. `escala = mínimo entre escala y max_leverage` (el infinito se queda en `max_leverage`)
6. Donde la escala siga siendo NaN (calentamiento), `escala = 0`.
7. Devuelve `posiciones × escala`, elemento a elemento.

Pistas.

Pista 1 · Dos problemas separados

Primero estima el riesgo con la información disponible al decidir la posición (hasta `t-1`); después conviértelo en una escala con su tope y su calentamiento. Las posiciones solo intervienen al final, en una multiplicación.

Pista 2 · Herramientas

`rolling(lookback, min_periods=lookback).std()` calcula la desviación típica móvil de cada columna con `ddof = 1` por defecto. `shift` mueve una tabla hacia fechas posteriores. `clip(upper=...)` aplica un tope y deja los NaN como NaN. `fillna(0.0)` rellena. `np.sqrt(periods)` anualiza. Dos DataFrames con el mismo índice y las mismas columnas se multiplican elemento a elemento.

Pista 3 · El orden importa

En pandas, dividir entre 0 da `inf` sin lanzar ninguna excepción, y el tope lo convierte en `max_leverage`: no hace falta tratar $\hat{\sigma} = 0$ aparte. Pero si rellenas con 0 los NaN del calentamiento antes de dividir, cada 0 se convierte en `inf` y después en `max_leverage`, y el calentamiento deja de valer 0: rellena al final. El desplazamiento se aplica a la estimación de volatilidad (o a la escala), nunca a las posiciones, que ya vienen desplazadas.

4. Errores típicos

Usar la volatilidad que incluye el día `t`

Sin desplazar la estimación, la escala del día `t` usa `r_t`, que no se conocía al decidir la posición. Síntoma: el backtest mejora un poco "gratis", porque la posición se reduce justo los días de grandes movimientos. Lo detecta `test_apply_vol_target_uses_only_past_returns`.

Desplazar dos veces

Si desplazas la estimación dos filas, la escala llega un día tarde. Síntoma: la fila `lookback + 1` todavía vale 0. Lo detecta `test_apply_vol_target_warmup_is_zero`. Desplazar también las posiciones es el mismo error (la señal llega con dos días de retraso): no lo hagas, ya vienen desplazadas. Lo detecta `test_apply_vol_target_exact_scale`, que usa posiciones que alternan `+1` y `-1`.

Calentamiento demasiado corto

Con `min_periods=2`, por ejemplo, la primera estimación sale de dos rendimientos y es puro ruido. Síntoma: escalas distintas de 0 en las primeras filas. Lo detecta `test_apply_vol_target_warmup_is_zero`.

Rellenar con 0 antes de dividir

$0,10 / 0 = \text{inf}$, el tope lo deja en `max_leverage` y el calentamiento entero queda apalancado. Síntoma: las primeras filas valen 2 en vez de 0. Lo detecta `test_apply_vol_target_warmup_is_zero`.

Olvidar el tope o el relleno

Sin tope, un activo con $\hat{\sigma} = 0$ recibe una posición infinita; sin relleno, el calentamiento queda en NaN. En ambos casos `run_backtest` lanza `ValueError` con el mensaje "positions contiene NaN o infinitos". Lo detectan `test_apply_vol_target_caps_leverage` y `test_run_backtest_with_vol_target`.

Desviación típica de numpy

`np.std` usa `ddof = 0`. Con $L = 60$ la diferencia es inferior al 1 % y no se nota en la volatilidad realizada, pero no es la convención del docstring ni la de `metrics.py`. `test_apply_vol_target_exact_scale` compara la escala fila a fila con $L = 5$, donde la diferencia es del 12 %, y lo detecta. Si trabajas con arrays de numpy, pasa `ddof=1`.

5. Preguntas y ejercicios

1. ¿Por qué el Sharpe no cambia si multiplicas todos los rendimientos por 0,5? Entonces, ¿de dónde puede salir una mejora del Sharpe con vol targeting?
2. Calcula a mano la escala para $\hat{\sigma} = 25\%$ y para $\hat{\sigma} = 5\%$, con objetivo 10 % y tope 2.
3. ¿Qué ganas y qué pierdes con `lookback = 20` frente a `lookback = 120`? Piensa en reactividad, ruido de la estimación y turnover.
4. Con 10 activos al 10 % cada uno y correlación media 0,3, ¿qué volatilidad esperas de la cartera equiponderada? ¿Y si la estrategia solo está invertida en la mitad de los activos?
5. Cuando la escala supera 1, ¿qué coste falta en el modelo? Estímalo para una escala media de 1,3 y un tipo de financiación del 4 % anual.
6. **Ejercicio (notebook 02, apartado 7).** Compara `buy_and_hold` y `sma_crossover` con y sin `vol_target=0.10` usando `summary_table`. ¿Qué cambia más: el CAGR, la volatilidad, el drawdown o el Sharpe? Relaciónalo con la pregunta 1.
7. **Ejercicio (notebook 02).** Para un activo, por ejemplo SPY, dibuja la volatilidad móvil de 60 días de su P&L con y sin escalar (posiciones por `asset_returns` del `BacktestResult`). Comprueba que la escalada ronda el 10 % y localiza los episodios en los que se aleja. ¿Por qué justo ahí?
8. **Ejercicio (notebook 02).** Mide el turnover anual de buy-and-hold con y sin vol targeting con `annual_turnover`. ¿Cuadra con el orden de magnitud del apartado 2?

6. Checkpoint

```
python -m pytest -m extension           # solo los tests de la extensión: los 7 de
    este capítulo
python -m pytest tests/test_vol_target.py # lo mismo, por fichero
python -m pytest -k apply_vol_target     # los 6 tests de apply_vol_target
python -m pytest -k apply_vol_target -rs # con el motivo de cada skipped
python -m pytest -m "not extension"      # todo el proyecto salvo la extensión
```

Antes de empezar, `python -m pytest -m extension` da **7 skipped**, con el motivo "TODO pendiente: TODO 6.1 · apply_vol_target" y la sección "TODO pendientes" al final. Al terminar, **7 passed**. El séptimo test, `test_run_backtest_with_vol_target`, pasa por `run_backtest`: si sale skipped con otro TODO (1.1 o 2.x), resuélvelo primero. Un TODO implementado pero incorrecto sale como **FAILED** con el mensaje del `assert`. Ten en cuenta que un `python -m pytest` sin filtros también ejecuta estos tests: mientras no hagas la extensión, aparecen como skipped, no como fallos; si no la vas a hacer, `-m "not extension"` los deja fuera.